

```
In [1]: # This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list the files in the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets mounted as /kaggle/working
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

/kaggle/input/imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv

```
In [2]: import pandas as pd
df = pd.read_csv('/kaggle/input/imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv')
```

```
In [3]: df = df.drop_duplicates()
df.duplicated().sum()
```

Out[3]: 0

```
In [4]: df.shape
```

Out[4]: (49582, 2)

```
In [5]: import nltk
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to /usr/share/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /usr/share/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /usr/share/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package wordnet to /usr/share/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

Out[5]: True

```
In [6]: # POS Mapper
from nltk import word_tokenize, pos_tag
from nltk.corpus import stopwords, wordnet
def get_wordnet_pos(word):
    tag = pos_tag(word)[0][1].upper()
    tag_dict = {
        'J': wordnet.ADJ,
        'N': wordnet.NOUN,
```

```

        'V': wordnet.VERB,
        'R': wordnet.ADV
    }
    return tag_dict.get(tag, wordnet.NOUN)

```

```

In [7]: import re
        from bs4 import BeautifulSoup
        from nltk.stem import WordNetLemmatizer

        stop_words = set(stopwords.words('english'))
        lemmatizer = WordNetLemmatizer()

        def preprocess_text(text):
            # Remove HTML tags
            soup = BeautifulSoup(text, 'lxml')
            text = soup.get_text(separator=' ')

            # Remove URLs
            text = re.sub(r'http\S+|www\S+|https\S+', '', text)

            # Word Tokenization
            tokens = word_tokenize(text.lower())

            # Remove stopwords + non-alphabetic + Lemmatize with POS
            tokens = [
                lemmatizer.lemmatize(w, get_wordnet_pos(w))
                for w in tokens if w.isalpha() and w not in stop_words
            ]

            # Join the tokens back into a string
            return ' '.join(tokens)

```

```

In [8]: df['cleaned_review'] = df['review'].apply(preprocess_text)

```

```

In [9]: from collections import Counter
        import matplotlib.pyplot as plt
        # Visualize the most frequent words per category using bar plots
        positive_words = ' '.join(df[df['sentiment']=='positive']['cleaned_review'])
        negative_words = ' '.join(df[df['sentiment']=='negative']['cleaned_review'])

        def most_common_words(text, title, n):
            if not text:
                print(f'No words found for {title}')
                return

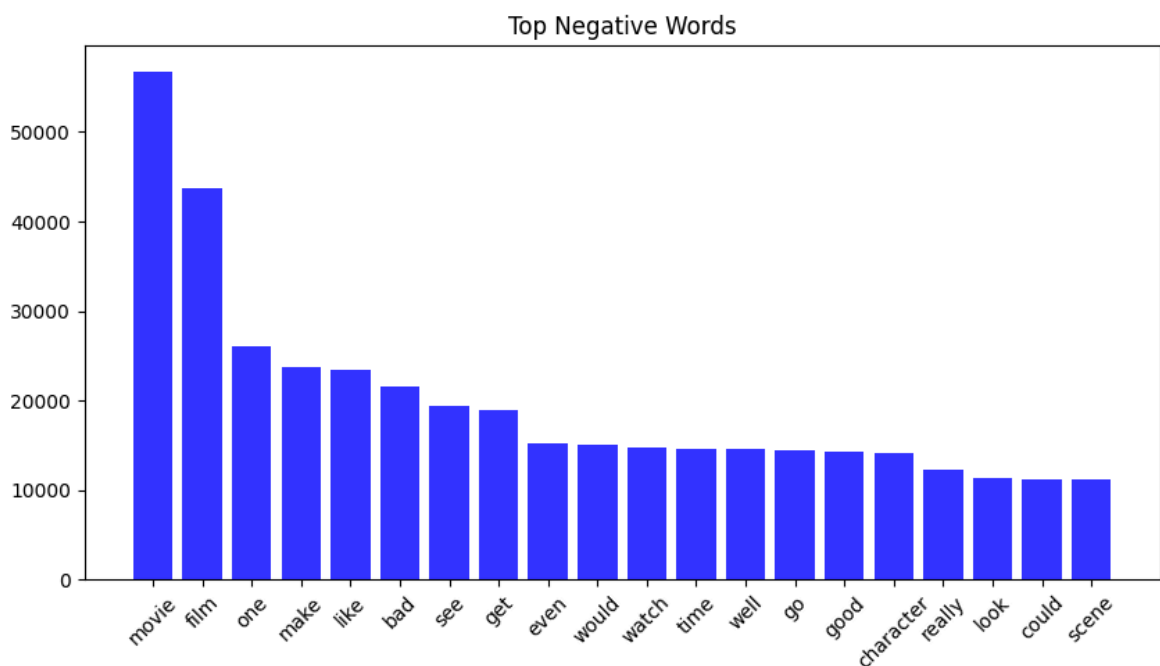
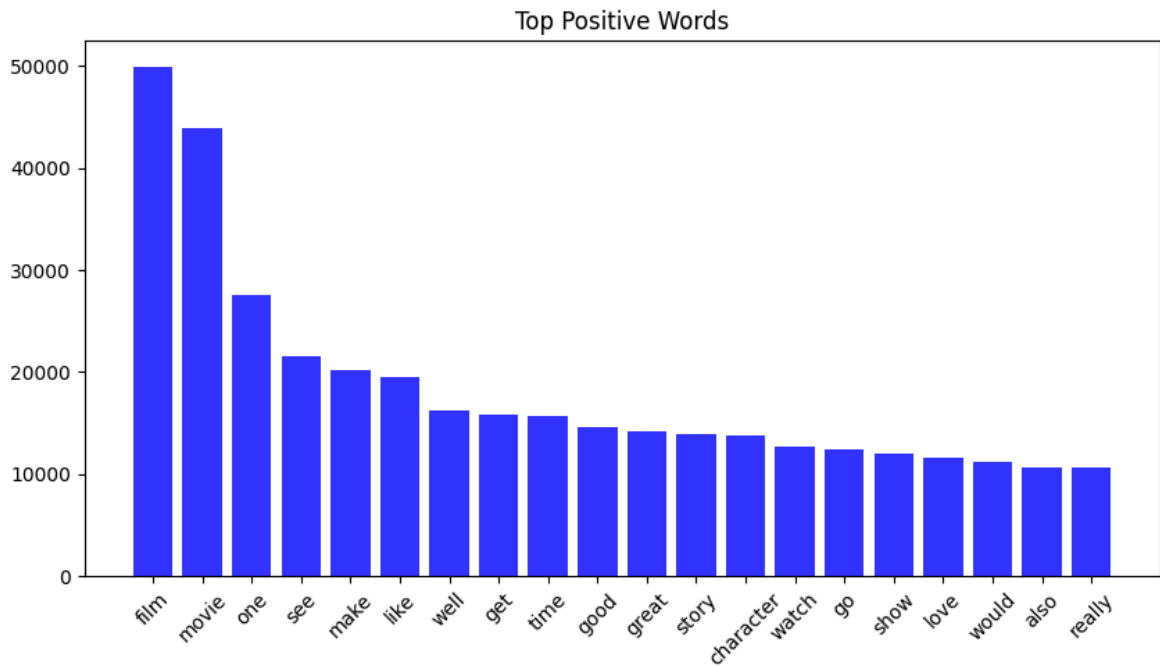
            words = text.split() # Split the text into words
            counter = Counter(words)
            common = counter.most_common(n) # Get the top n most common words

            if not common:
                print(f'No words found for {title}')
                return

            words, counts = zip(*common)
            plt.figure(figsize=(10,5))
            plt.bar(words, counts, color='#33f')
            plt.title(title)
            plt.xticks(rotation=45)
            plt.show()

```

```
most_common_words(positive_words, 'Top Positive Words', 20)
most_common_words(negative_words, 'Top Negative Words', 20)
```



```
In [10]: # Score Distribution
plt.figure(figsize=(8, 4))
df['sentiment'].value_counts().plot(kind='bar', color=['#3f6', '#f33'])
plt.xlabel('Review Score')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.title('Score Distributions')
plt.tight_layout()
plt.show()
```



```
In [11]: # Visualize the most frequent words per category using word clouds
from wordcloud import WordCloud
positive_WordCloud = WordCloud(width=800, height=400, background_color='white',
negative_WordCloud = WordCloud(width=800, height=400, background_color='white',

plt.figure(figsize=(15,10))

plt.subplot(1,2,1)
plt.imshow(positive_WordCloud, interpolation='bilinear')
plt.title('Most frequent Positive words')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(negative_WordCloud, interpolation='bilinear')
plt.title('Most frequent Negative words')
plt.axis('off')

plt.tight_layout()
plt.show()
```



```
In [12]: data = df.copy()
data.shape
```

```
Out[12]: (49582, 3)
```

```
In [13]: # Train-Test Split
from sklearn.model_selection import train_test_split
X = data['cleaned_review']
y = data['sentiment']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

```
In [14]: print("Train set:")
print(y_train.value_counts(normalize=True) * 100)
print("\nTest set:")
print(y_test.value_counts(normalize=True) * 100)
```

Train set:
sentiment
positive 50.187823
negative 49.812177
Name: proportion, dtype: float64

Test set:
sentiment
positive 50.186548
negative 49.813452
Name: proportion, dtype: float64

```
In [15]: from sklearn.model_selection import GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, accuracy_score
# Set Pipeline
pipe = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('clf', LogisticRegression(solver='saga', max_iter=1000, random_state=23))
])

# Grid
param_grid = {
    'tfidf__ngram_range': [(1,1), (1,2)],
    'tfidf__max_df': [0.7, 0.8, 0.9],
    'tfidf__min_df': [3, 5],
    'clf__C': [0.1, 1, 10],
    'clf__penalty': ['l2'],
}

# Create GridSearch
grid = GridSearchCV(pipe, param_grid, cv=3, scoring='accuracy', n_jobs=-1, verbose=1)

# Train Model
grid.fit(X_train, y_train)
```

Fitting 3 folds for each of 36 candidates, totalling 108 fits

```
Out[15]: GridSearchCV
  estimator: Pipeline
    TffidfVectorizer
    LogisticRegression
```

Best Hyperparameters

```
{
    'clf__C': 10,
    'clf__penalty': 'l2',
```

```

'tfidf__max_df': 0.7,
'tfidf__min_df': 3,
'tfidf__ngram_range': (1, 2)
}

```

```

In [16]: # Predict Test Set
y_pred = grid.predict(X_test)

# Accuracy
from sklearn.metrics import accuracy_score
print("Test Accuracy:", accuracy_score(y_test, y_pred))

# Classification Report (precision, recall, f1-score)
from sklearn.metrics import classification_report
print("Classification Report:\n", classification_report(y_test, y_pred))

```

Test Accuracy: 0.9017848139558334

Classification Report:

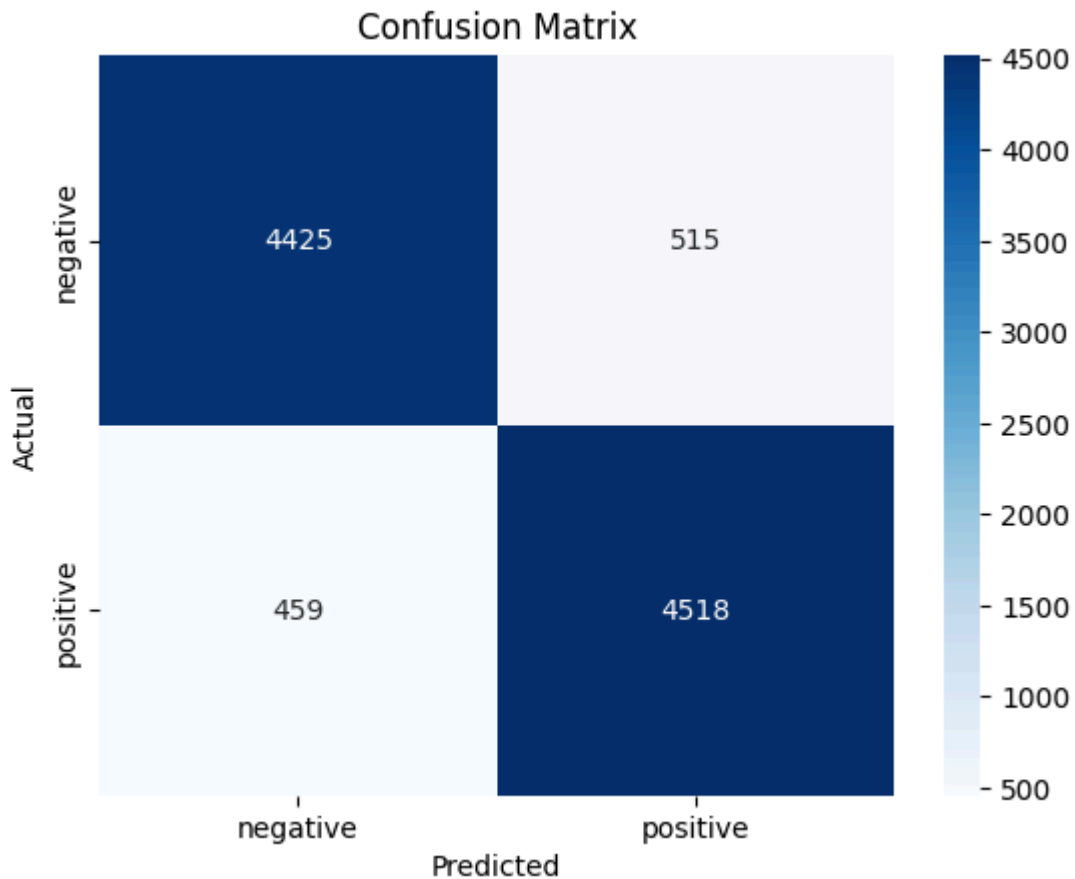
	precision	recall	f1-score	support
negative	0.91	0.90	0.90	4940
positive	0.90	0.91	0.90	4977
accuracy			0.90	9917
macro avg	0.90	0.90	0.90	9917
weighted avg	0.90	0.90	0.90	9917

```

In [17]: from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred, labels=grid.classes_)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=grid.classes_, yticklabels=grid.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```



```
In [18]: import numpy as np
feature_names = grid.best_estimator_.named_steps['tfidf'].get_feature_names_out()
coefficients = grid.best_estimator_.named_steps['clf'].coef_[0]

# show top positive & top negative
top_positive = np.argsort(coefficients)[-10:]
top_negative = np.argsort(coefficients)[:10]

print("Top positive words:", feature_names[top_positive])
print("Top negative words:", feature_names[top_negative])
```

Top positive words: ['today' 'favorite' 'best' 'amaze' 'brilliant' 'wonderful' 'hilarious' 'perfect' 'great' 'excellent']
 Top negative words: ['bad' 'waste' 'awful' 'boring' 'poor' 'nothing' 'terrible' 'horrible' 'dull' 'disappointment']

```
In [19]: predictions = grid.predict(["The best movie"])
print(predictions)
```

['positive']

```
In [20]: import joblib

joblib.dump(grid.best_estimator_, 'sentiment_model.pkl')
```

Out[20]: ['sentiment_model.pkl']

```
In [21]: model = joblib.load('sentiment_model.pkl')
pred = model.predict(["The best movie"])
print(pred)
```

['positive']